



《实用Python编程》

第13讲：大模型辅助编程

实用Python编程 教研组



从想法到代码

- 大模型辅助编程，帮助初学者越过两个关键门槛
 - 建立计算思维：如何将模糊、抽象的业务需求拆解为计算机能够执行的算法步骤？
 - 步骤变代码：如何将思路描述变为可执行的代码？

任务

从一个文档中
找出高频词



算法步骤

Step 1. 读取文档
Step 2. 拆分文档成词序列
Step 3. 遍历词序列，统计词频
Step 4. 排序并输出高频词

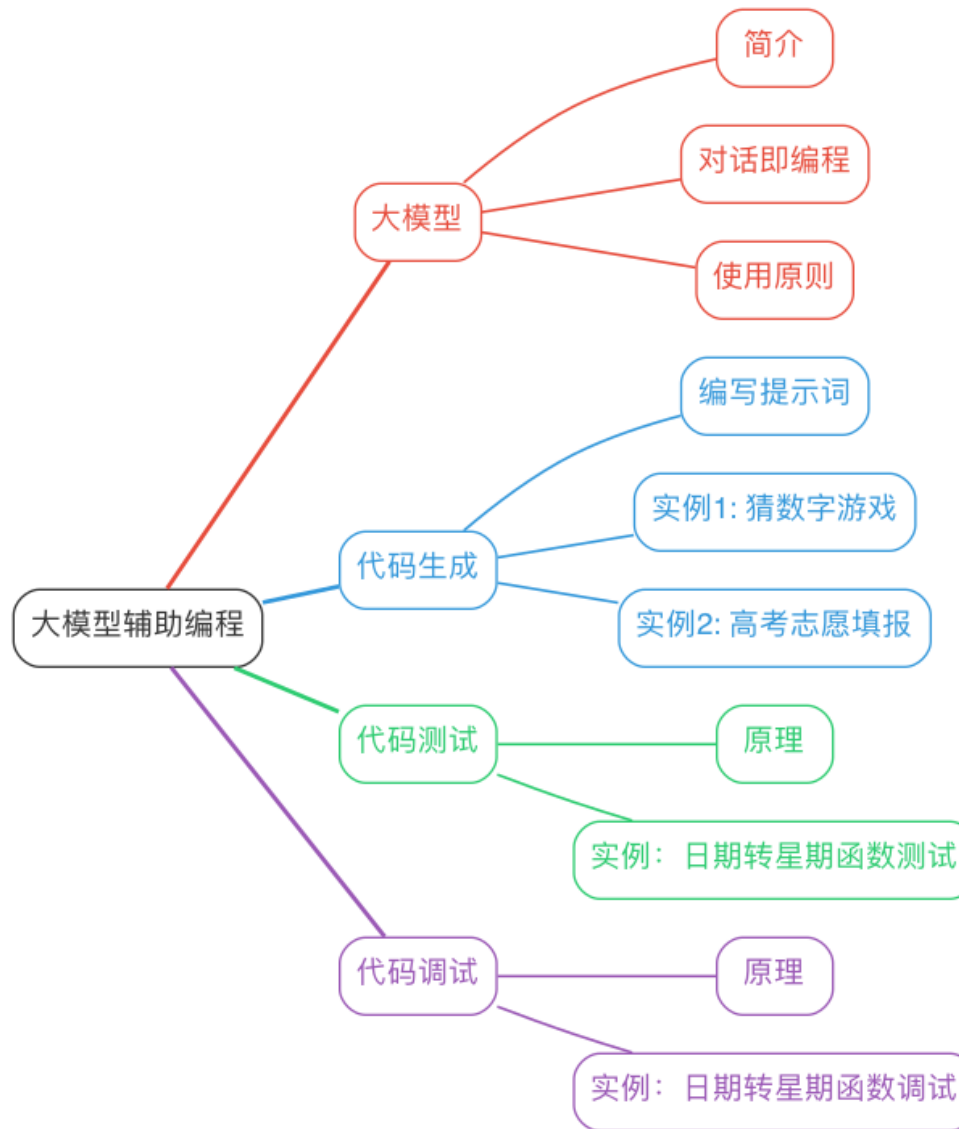


代码实现

```
lines = open('data.txt').readlines()
cnt = Counter()
for line in lines:
    for word in line.split():
        cnt[word] += 1
print(cnt.most_common(20))
```

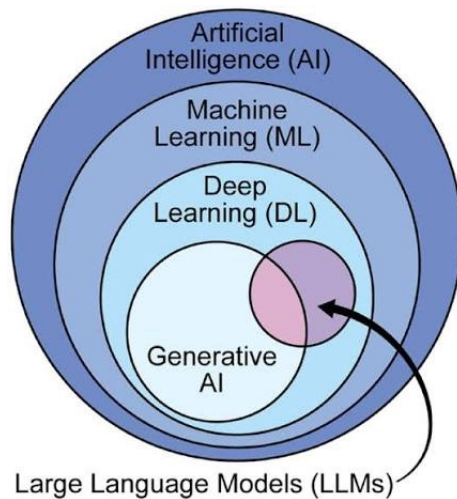
知识点：文件I/O、计数器、嵌套循环、字符串操作、...

提纲

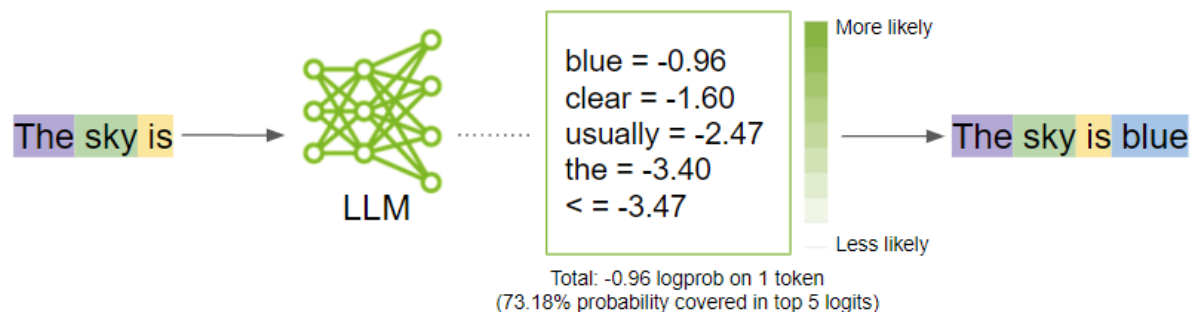


大模型

- 以GPT为代表的大语言模型 (large language models, LLMs)
- 本质上是一种基于深度学习构建的复杂人工智能系统
 - 通过**学习**海量文本数据来理解和**生成**人类语言

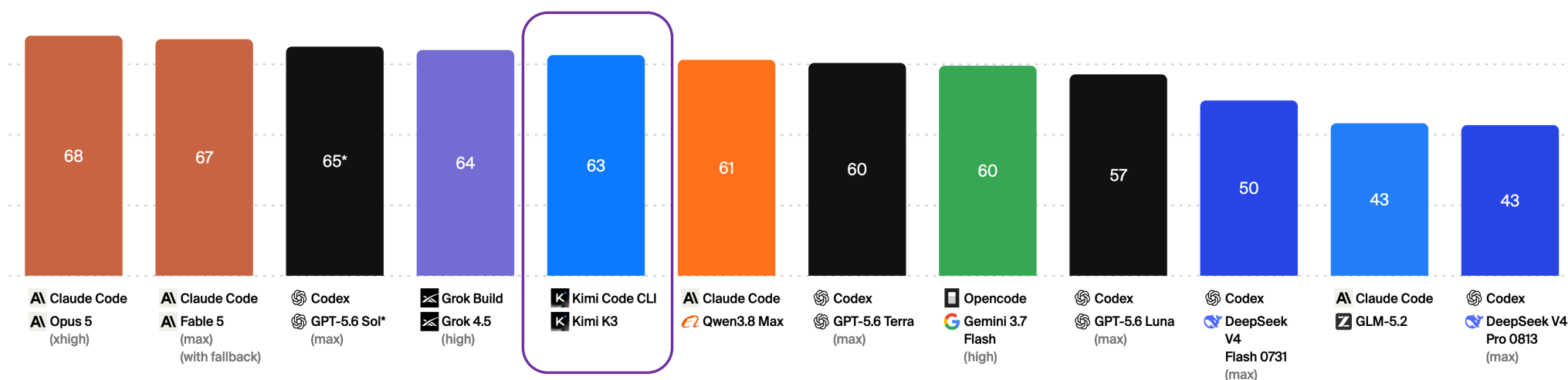


LLM通过不断预测下一个词元 (token) 来生成回答



大模型编程能力评测

- 综合考虑模型性能与访问便捷性，我们选择Kimi作为教学案例
 - 由于大模型快速发展，课件中展示的案例可能与教材有出入



根据三个评测集 (DeepSWE, Terminal-Bench v2.1, SWE-Atlas-QnAd) 平均分给出的大模型编程能力榜单 (2026年8月)
<https://artificialanalysis.ai/agents/coding-agents>



对话即编程 (chat as coding)

如何用Python在一堆数中找出中位数

- 要让大模型变成编程助手，最简便、直观的形式就是与其对话
 - 依托于LLM的自然语言理解与生成能力
 - 对初学者而言，大模型就像一位24小时在线、知识广博、经验丰富但偶尔也会犯错的程序员
- **提示词 (prompt)**
 - 用户发送给大模型的指令

用好大模型可极大缩短学习曲线、提高编程效率

在 Python 里找“一堆数”的中位数，最省事的做法就是直接用标准库，两步搞定：

Python

复制

```
import statistics as st

data = [7, 1, 3, 5, 9, 2, 8]
median_val = st.median(data) # 结果 5
```

statistics.median 会自动：

1. 把数据排序
2. 判断长度奇偶，奇数取中间，偶数取中间两个的均值

如果你用的是 NumPy 数组（尤其是数据量很大时），可以用：

Python

复制

```
import numpy as np
data = np.array([7, 1, 3, 5, 9, 2, 8])
median_val = np.median(data) # 结果 5.0
```

np.median 内部用“部分快排”算法，不需要完整排序，速度更快，内存更省。



对话即编程

- 大模型作为编程助手的四种角色

角色	功能	提示词示例
老师	编程概念讲解	请结合例子解释Python的浅copy与深copy的区别
打字员	代码生成与补全	我想用Python写一个命令行倒计时器，每秒打印剩余时间
医生	代码问题诊断与调试	运行时报错IndexError，这是我的代码[粘贴代码]，为什么错了并帮我修复
评论员	代码解释与重构	这段代码太乱了，如何写得更优雅、更符合Python规范？



使用原则：大模型是辅助，不是替代

- **不可取的做法**

- 把自己定位为只做“复制粘贴”的搬运工
- 直接把题目扔给大模型，然后复制回答交差

- **推荐的用法**

- 先让大模型解释思路，通过与它对话培养自己的计算思维
- 大模型生成代码后逐行阅读，遇到不懂的语句立刻追问
- 通过手敲一遍、或尝试修改/优化部分代码，锻炼自己的编程能力

利用大模型生成代码

• 编写高效的代码生成提示词

- 生成代码的质量，很大程度上取决于如何在提示词中描述任务
- 提示词好比当前任务的“说明书”，可从四个维度编写要素齐全的提示词

要素	作用	表达示例
角色设定	指定大模型以何种身份完成任务	你是一个数据分析专家，擅长使用Python分析和处理各种数据。
任务描述	明确核心功能与处理步骤	读取TXT文件，统计每个单词出现次数，按频率从高到低排序输出。
约束与要求	边界条件、库依赖、注释程度等限制	不依赖第三方库；包含必要注释；处理文件不存在等异常情况。
输入输出示例	用具体示例减少歧义	示例输入text.txt: Python is great, I love Python. 输出: {"python": 2, ...}"



实例1：猜数字游戏

- 猜数字游戏（“数字炸弹”）
 - 程序想一个数字（1~1000），人来猜
- 推荐做法
 - 先让大模型给出编程思路，自己尝试编写代码
 - 再对比手写代码与生成代码的差异，体会其中的差距

思路要点

random生成随机数 → while循环反复猜 → 比较大小给出提示 → 记录猜测次数



凝练成一段包含角色设定、任务描述、约束与要求等多维度描述的结构化提示词

我是 Python 初学者，想写一个“猜数字”的小游戏。规则是电脑随机生成 1~100 的数字，我来猜，系统提示猜大了还是猜小了。请告诉我大概的编写思路，先不要直接给完整代码。

思路可以拆成 5 步，每一步都只需要你学过的最基础的语法就能搞定：

- 让电脑“想好”一个 1~100 的整数
用 `import random` 之后的一行就能完成，这个值存到一个变量里，全程不变。
- 让用户反复输入猜测
用 `while True` 制造“无限”循环；在循环里用 `input()` 拿到用户敲的数字，记得用 `int()` 转成整数。
- 把用户的数和电脑想的数比大小
用普通的 `if / elif / else` 就能给出三条分支：
 - 猜对了 → 输出恭喜并结束循环（`break`）
 - 猜大了 → 提示“大了”
 - 猜小了 → 提示“小了”
- 让游戏更友好一点（可选）
 - 捕获用户手滑输入非数字的情况：用 `try...except ValueError` 包住 `int()`。
 - 记录猜的次数，猜完后告诉用户“你一共猜了 N 次”。
- 收尾
猜对后跳出循环，打印结束语。整个程序不到 30 行就能跑起来。

先按这 5 步自己写，遇到问题再把具体报错贴出来，我再帮你逐一解决。



实例1：猜数字游戏

- 对照提示词阅读代码
 - 三条约束（提示不含答案、简洁、Notebook可运行）是否都满足？

运行效果（人机交互）

请输入1~1000之间的整数：500
大了
请输入1~1000之间的整数：250
小了
请输入1~1000之间的整数：375
恭喜，猜对了！共猜了8次。

提示词（表7.3）

你是一个Python程序员。（角色设定）
请根据下列步骤实现一个猜数字游戏的程序。（任务描述）
1. 让电脑随机想一个1~1000（含边界）之间的整数。
2. 让用户反复输入猜测。猜对输出"恭喜"结束；否则给出提示。
3. 游戏结束时，输出用户一共猜了多少次。
注意事项：（约束与要求）
1. 给出的提示不能直接包含答案。
2. 代码尽量简洁、易懂，不使用第三方工具包。
3. 代码要能在Jupyter Notebook中运行。

示例代码7.1（大模型生成）

```
# 猜数字游戏（人猜电脑想）
import random
target = random.randint(1, 1000) # 电脑想的数
tries = 0 # 计数器
while True:
    guess = int(input("请输入1~1000之间的整数："))
    tries += 1
    if guess == target:
        print(f"恭喜，猜对了！共猜了{tries}次。")
        break
    print("大了" if guess > target else "小了")
```

括号中的文字仅用于说明，并非提示词的一部分



实例1：猜数字游戏

- 逐行阅读大模型生成的代码，并与自己手搓方案比较，加深理解

示例代码7.1 (大模型生成)

```
# 猜数字游戏 (人猜电脑想)
import random
target = random.randint(1, 1000) # 电脑想的数
tries = 0                        # 计数器
while True:
    guess = int(input("请输入1~1000之间的整数: "))
    tries += 1
    if guess == target:
        print(f"恭喜, 猜对了! 共猜了{tries}次。")
        break
    print("大了" if guess > target else "小了")
```

- `random.randint(1, 1000)`: 生成1~1000 (含边界) 的随机整数
- `while True + break`: 反复猜测, 猜对时跳出循环
- f-string中的`{tries}`: 实时输出猜测次数
- "大了" `if guess > target` else "小了": 条件表达式给出提示, 且不泄露答案

如果输入不是整数会怎样? 如何让程序更健壮?



课堂练习-7-1

- 在大模型辅助下实现一个网页版的猜数字游戏
 - 基于Streamlit框架 (6.5节)

猜数字游戏 (1-1000)

再来一局

输入你的猜测:

651

猜!

小了

对于来自真实场景的复杂任务，代码生成往往不是一蹴而就的



实例2：高考志愿填报推荐系统

- **任务需求（4.5节案例的升级）**
 - 以2024年录取数据为参考，为2025年北京考生推荐可报考的院校和专业组
 - 交互式Web界面：输入位次 + 选考科目
 - 输出"冲击/稳妥/保底"三个区间的专业组
- **开发流程全部由大模型辅助完成**
 - 抓取网页数据 → 清洗数据 → 查询与结果展示

北京高考志愿推荐系统

基于2024年录取数据，为2025年考生提供志愿推荐

请输入您的市排名（位次）

16470

请选择您的选考科目（可不选）

物理 × 化学 × 生物 ×

生成推荐方案

您的位次: 16,470 | 调整后等效2024年位次: 14,027

🚀 冲一冲

冲击院校，录取概率约30-50%

中央美术学院

专业组: 1 | 选考科目: 不限(中外合办) | 位次:12095 | 投档线: 593分

辽宁大学

专业组: 4 | 选考科目: 物理 + 化学 | 位次:12095 | 投档线: 593分

南京师范大学

专业组: 3 | 选考科目: 物理 + 化学 | 位次:12095 | 投档线: 593分

✅ 稳一稳

稳妥选择，录取概率约70-80%

北京语言大学

专业组: 2 | 选考科目: 物理 | 位次:13407 | 投档线: 586分

大连海事大学

专业组: 4 | 选考科目: 物理 | 位次:13407 | 投档线: 586分

南京审计大学

专业组: 1 | 选考科目: 不限(中外合办) | 位次:13407 | 投档线: 586分

🛡️ 保一保

保底院校，录取概率>90%

首都医科大学

专业组: 9 | 选考科目: 物理 + 化学 | 位次:14812 | 投档线: 579分

东北电力大学

专业组: 1 | 选考科目: 物理 + 化学(中外合办) | 位次:14812 | 投档线: 579分

云南大学

专业组: 1 | 选考科目: 物理 | 位次:14812 | 投档线: 579分



实例2：高考志愿填报推荐系统

• Step 1. 抓取网页数据

表 7.4: 用于生成高考数据爬取代码的提示词

你是一个网络爬虫专家，擅长使用 Python 进行网页数据爬取。（角色设定）

请你为我提供一个静态网页的 Python 爬虫代码。需要爬取的两个网站的地址是 https://gaokao.eol.cn/bei_jing/dongtai/202407/t20240720_2625168.shtml 和 https://gaokao.eol.cn/bei_jing/dongtai/202406/t20240625_2619192.shtml，将每个网站都保存成一个 html 文件。（任务描述）

将网页爬虫组织成一个类，方便重复调用。在确保代码正确的前提下，代码尽可能简洁高效（最好不超过 50 行代码），且要包含必要注释。（约束和要求）

对比人工编写的PageDownloader（3.3.2节），大模型给出的StaticPageCrawler合并了部分方法（如crawl与save），代码更紧凑

在命令行执行python crawler.py，抓取投档线与分段表网页并保存为 gaokao_page_1.html、gaokao_page_2.html

- ✓ 成功保存: https://gaokao.eol.cn/bei_jing/dongtai/202407/t20240720_2625168.shtml → gaokao_page_1.html
- ✓ 成功保存: https://gaokao.eol.cn/bei_jing/dongtai/202406/t20240625_2619192.shtml → gaokao_page_2.html

从对话框中复制并保存为crawler.py

```
1 import requests
2 from pathlib import Path
3
4 class StaticPageCrawler:
5     """静态网页爬虫类"""
6     def __init__(self, timeout=10):
7         self.timeout = timeout
8         self.headers = {
9             'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
10        }
11    def crawl_and_save(self, url, output_path):
12        """爬取网页并保存为HTML文件"""
13        try:
14            response = requests.get(url, headers=self.headers, timeout=self.timeout)
15            response.raise_for_status()
16            response.encoding = response.apparent_encoding # 自动检测编码
17            Path(output_path).write_text(response.text, encoding='utf-8')
18            print(f"✓ 成功保存: {url} → {output_path}")
19            return True
20        except Exception as e:
21            print(f"x 爬取失败: {url} - {str(e)}")
22            return False
23
24 # 使用示例
25 if __name__ == "__main__":
26     urls = [
27         "https://gaokao.eol.cn/bei_jing/dongtai/202407/t20240720_2625168.shtml",
28         "https://gaokao.eol.cn/bei_jing/dongtai/202406/t20240625_2619192.shtml"
29     ]
30     crawler = StaticPageCrawler()
31     for i, url in enumerate(urls, 1):
32         crawler.crawl_and_save(url, f"gaokao_page_{i}.html")
```



实例2：高考志愿填报推荐系统

- Step 2. 清洗数据
 - 观察原始数据，在提示词中给出清洗规则

2024年北京高考分段表

分数	人数	累计人数
700-750	117	117
699	15	132
698	16	148
524	226	26772
523 (特招线)	224	26996
436	149	44652
435	137	44789
434 (本科线)	134	44923
433	130	45053
432	161	45214

北京市2024年高招本科普通批录取投档线公布，详细如下：

	院校	专业组	总分
1021	北京大学	2 不限	700
1023	清华大学	2 物理	698
1021	北京大学	3 物理+化学	690
1023	清华大学	3 物理+化学	689
1021	北京大学	1 不限	688
1023	清华大学	1 不限	688
3112	上海交通大学	3 物理+化学	687

表 7.5: 用于生成高考数据清洗代码的提示词

你是一个数据清洗专家，擅长使用 Python 进行表格数据清洗。（角色设定）

请你为我提供一个 html 网页文件中的表格数据的 Python 清洗代码，需要清洗的数据是投档线数据和分段表数据两个表格，具体需求为：

对分数段数据：

1. 跳过原表第一行，并重置行索引
2. 将列名重命名为“分数”、“人数”、“位次”
3. 将第一行的高分段（“700-750”）简化为“700”
4. 定位本科线并舍弃该行之后的数据
5. 通过正则匹配清除个别分数单元格中包含的注解（特招线、本科线等）

对投档线数据：

1. 跳过原表第一行，并重置行索引
2. 将列名重命名为“院校代码”、“院校”、“专业组代码”、“限制”、“投档线”
3. 利用分段表提供的分数到位次映射，将投档线数表中的每个投档线都转成对应位次，并作为新的一列添加到投档线表（任务描述）

直接使用 pandas 的 read_html 函数读取 html 文件中的表格数据；

输入为两个 html 文件 (gaokao_page_1.html 和 gaokao_page_2.html)，每个 html 包含一个待清洗的表格，gaokao_page_1.html 中的是投档线表格，gaokao_page_2.html 中的是分段表表格；

输出为两个清洗好的表格 csv 文件；

在确保代码正确的前提下，代码尽可能简洁高效（最好不超过 50 行），且要包含必要注释。（约束和要求）



实例2：高考志愿填报推荐系统

• 大模型生成的数据清洗代码

```
1 import pandas as pd
2 # 读取并清洗分段表
3 df_score = pd.read_html('gaokao_page_2.html')[0].iloc[1:].reset_index(drop=True)
4 df_score.columns = ['分数', '人数', '位次']
5 # 定位并删除本科线之后的行 (假设本科线行包含文本标注)
6 undergrad_idx = df_score[df_score['分数'].astype(str).str.contains('本科线')].index
7 if len(undergrad_idx):
8     df_score = df_score.iloc[:undergrad_idx[0]+1]
9 # 提取分数数字, 处理第一行特殊情况
10 df_score['分数'] = df_score['分数'].astype(str).str.extract(r'(\d+)')[0]
11 df_score.loc[0, '分数'] = '700' # 将"700-750"简化为"700"
12 df_score['分数'] = pd.to_numeric(df_score['分数'])
13 # 创建分数-位次映射字典
14 score_to_rank = dict(zip(df_score['分数'], df_score['位次']))
15 # 读取并清洗投档线表
16 df_admission = pd.read_html('gaokao_page_1.html')[0].iloc[1:].reset_index(drop=True)
17 df_admission.columns = ['院校代码', '院校', '专业组代码', '限制', '投档线']
18 df_admission['投档线'] = pd.to_numeric(df_admission['投档线'].astype(str).str.extract(r'(\d+)')[0])
19 # 添加位次列 (通过映射字典)
20 df_admission['位次'] = df_admission['投档线'].map(score_to_rank)
21 # 输出CSV文件
22 df_score.to_csv('cleaned_score_segments.csv', index=False, encoding='utf-8-sig')
23 df_admission.to_csv('cleaned_admission_lines.csv', index=False, encoding='utf-8-sig')
```

关键语句解读

- L3: read_html: 直接读取HTML文件中的表格
- L3: iloc[1:] + reset_index: 跳过原表首行
- L10: str.extract(r"(\d+)"): 正则提取纯数字
- L14: zip + dict: 构建"分数→位次"映射
- L20: map: 为投档线批量换算位次

保存为clean_data.py

在命令行中运行python clean_data.py

获得清洗后数据cleaned_admission_lines.csv

院校代码, 院校, 专业组代码, 限制, 投档线, 位次
1021, 北京大学, 2, 不限, 700, 117
1023, 清华大学, 2, 物理, 698, 148
1021, 北京大学, 3, 物理+化学, 690, 354
1023, 清华大学, 3, 物理+化学, 689, 384



实例2：高考志愿填报推荐系统

- 对复杂语句（如正则提取），可追问大模型逐项解释
- 读懂其他人的代码也有助于提升自己的编程能力

```
df_score['分数'] = df_score['分数'].astype(str).str.extract(r'(\d+)')[0]
```

请为我讲解一下这一行代码所做的操作



💡 思考已完成



这行代码是使用正则表达式从"分数"列中提取纯数字，清除单元格中的文字注解。我来逐步拆解：

代码分解

Python

📄 复制

```
df_score['分数'] = df_score['分数'].astype(str).str.extract(r'(\d+)')[0]
```

1. `astype(str)`：将"分数"列所有数据转为字符串类型，确保能使用字符串方法。
2. `.str.extract(r'(\d+)')`：核心操作，用正则提取数字。
 - `str`：访问字符串方法。
 - `extract()`：提取匹配正则表达式的内容。
 - `r'(\d+)'`：正则表达式（原始字符串格式）。
 - `\d`：匹配任意一个数字（0-9）。
 - `+`：匹配一个或多个前面的字符（此处表示连续数字）。
 - `()`：捕获组，表示提取这部分内容。
3. `[0]`：`extract()` 返回 DataFrame，取第 0 列（即捕获组提取的数字）。



实例2：高考志愿填报推荐系统

• Step 3. 搭建网页版的推荐系统

- 与之前不同，需将清洗后的csv文件以附件形式发送给大模型（私有数据要慎重）

表 7.6: 用于生成基于位次的志愿填报推荐程序的提示词

你是一个资深的 Python 程序员，擅长使用 Python 进行数据分析和可视化。（角色设定）

请你用 Python 帮我实现一个北京考生志愿推荐的可视化 demo，用户只需要提供自己的位次并选择自己的选考科目，系统便会推荐对应的专业组。

以 2024 年数据为参考，为 2025 年考生推荐可报考的院校和专业组。

已知 2024 年和 2025 年高考人数分别为 6.72 万和 7.89 万。

推荐将结合考生的位次和修读的科目，分出“冲击”、“稳妥”、“保底”三个区间，每个区间包含至多 10 个院校。

可冲击的专业组定义为当前排名往前 5% 至 15% 的专业组，较稳妥的专业组定义为当前排名前后 5% 的专业组，可保底的专业组定义为当前排名往后 5% 至 15% 的专业组。（任务描述）

使用 Streamlit 包来实现可视化 demo；

变量名应该规范命名，不要使用中文进行命名；

三个档位的专业组都展示在同一个页面中（3 列）；

在确保代码正确的前提下，代码尽可能简洁高效，如果生成的单个文件太长，需要按照逻辑拆分成多个文件，每个文件最好不要超过 50 行，且要包含必要注释。

在给出具体代码之前先展示各个文件的 tree 结构，以及是什么作用。（约束和要求）

附件是 2024 年的录取投档线及对应位次表格（输入输出示例）

定义推荐规则

就像写作一样，
应避免长难句，
因为hold不住



实例2：高考志愿填报推荐系统

- 大模型给出的代码框架
 - 按模块生成整个项目
 - 在命令行中执行 `streamlit run main.py` 即可启动系统

大模型给出的代码框架

```
project/
├── main.py          # Streamlit入口，页面布局与逻辑
├── config.py        # 配置参数（人数、区间系数等）
├── data_loader.py   # 数据加载与预处理
├── calculator.py    # 位次调整与区间计算逻辑
├── recommender.py   # 推荐算法实现
└── ui_components.py # UI组件渲染函数
```



实例2：高考志愿填报推荐系统

• 解读main.py

- 当用户点击“生成推荐方案”时，程序会依次调用各个子模块产生推荐结果并在界面中呈现

大模型给出的代码可用，
但有时候会显得比较笨拙

- df与输入无关，加载一次即可

```
7  def main():
8      st.set_page_config(page_title="北京高考志愿推荐系统", layout="wide")
9      st.title("北京高考志愿智能推荐")
10     st.caption("基于2024年录取数据为2025年考生提供参考")
11     # 用户输入区域
12     with st.container():
13         col1, col2 = st.columns(2)
14         with col1:
15             user_rank = st.number_input("请输入您的市排名（位次）", min_value=1, max_value=78900, value=5000, step=1)
16         with col2:
17             subject_options = ['物理', '化学', '生物', '政治', '历史', '地理']
18             selected_subjects = st.multiselect("请选择您的选考科目（可不选）", subject_options)
19     # 生成推荐
20     if st.button("生成推荐方案", type="primary"):
21         with st.spinner("正在分析数据..."):
22             # 加载数据
23             df = load_admission_data("cleaned_admission_lines.csv")
24             # 计算等效位次和区间
25             adjusted_rank = adjust_rank_for_2025(user_rank)
26             intervals = calculate_recommendation_intervals(adjusted_rank)
27             # 获取推荐
28             recommendations = get_recommendations(df, intervals, selected_subjects)
29             # 显示统计信息
30             st.info(f"您的位次: **{user_rank:,}** | 调整后等效2024年位次: **{adjusted_rank:,}**")
31             # 渲染结果
32             render_recommendation_section(recommendations)
33 if __name__ == "__main__":
34     main()
```



课堂练习-7-2

- 在大模型辅助下编写一个密码生成器
 - 密码需包含大写字母、小写字母、数字和特殊符号
 - 长度由用户输入
 - 如用户输入8，则输出形如aB3\$xY9!的随机密码

请对照表7.2自查，提示词四要素是否齐全？运行结果是否符合预期？

表 7.2: 用于代码生成的提示词结构要素

要素	作用	表达示例
角色设定	指定大模型以何种身份完成任务	你是一个数据分析专家，擅长使用 <i>Python</i> 分析处理各种数据。
任务描述	明确核心功能与处理步骤	读取一个包含多行英文单词的 <i>txt</i> 文件，统计每个单词出现次数，并按频率从高到低排序，以字典的形式输出。
约束与要求	指定边界条件、代码格式、库依赖、输入输出格式等限制条件	不允许依赖第三方库；代码需包含必要注释。处理文件不存在、内容为空等异常情况，并给出友好提示。
输入输出示例	用具体示例减少歧义，使生成的代码更匹配任务需求	示例输入文件 (<i>text.txt</i>): <i>Python is great, I love Python</i> . 示例输出: <code>{"python": 2, "is": 1, "great": 1, "i": 1, "love": 1}</code>



代码测试 vs. 代码调试

- 软件开发中两个至关重要的环节，利用大模型辅助可显著提高效率，发现人工检查不易察觉的潜在漏洞

比较维度	代码测试	代码调试
主要目标	发现问题（找出bug）	解决问题（消除bug）
过程性质	结构化的、预防性的（如编写测试用例）	探索性的、诊断性的（如查看日志）
时间节点	通常在程序发布或交付前	在测试发现错误或用户反馈报错后
主要工具	pytest	pdb



大模型辅助代码测试

- 原理：大模型擅长自然语言和代码的理解，能够从函数需求描述和实际代码中自动推断出需要覆盖的场景和潜在陷阱

基于Zeller公式实现的日期转星期几的函数

```
1 def date_to_weekday(date_str):
2     try:
3         year, month, day = map(int, date_str.split('-'))
4     except:
5         return "Invalid date format. Please use 'YYYY-MM-DD'."
6     if month < 3:
7         year -= 1
8         month += 12 #1月和2月变为上一年的13月和14月
9     q = day
10    m = month
11    k = year % 100
12    j = year // 100
13    h = (q + (13 * (m + 1)) // 5 + k + k // 4 + j // 4 + 5 * j) % 7
14    weekdays = ["Saturday", "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]
15    return weekdays[h]
```

- 功能：输入"YYYY-MM-DD"，返回对应星期几
- **人工检查代码隐患**
 - 只捕获split转换异常
 - 没有校验月/日是否越界
 - “1900-13-01”、“2023-02-30”也会给出似是而非的结果



大模型辅助代码测试

- Step 1. 准备提示词及待测试代码文件
 - 将待测试函数date_to_weekday存入mymodule.py
 - 作为附件上传给大模型，以便其分析
 - 在测试代码中通过from mymodule import date_to_weekday导入

 mymodule.py
651 Bytes

你是一个Python程序测试专家，善于生成各种测试用例，用于发现程序中的不完善之处。

请为附件中的date_to_weekday函数生成一组pytest风格的、全面的测试用例。要求尽可能多地覆盖边界条件、异常场景和正常功能。

最小日期设置为 1900 年 1 月 1 日。

注意观察提示词要素

- 角色设定：Python程序测试专家
- 任务描述：请为附件中的 ...
- 约束与要求：最小日期设置为 ...

下面给出针对 `date_to_weekday(date_str)` 的 **pytest** 风格的全面的测试用例，覆盖

1. 正常日期（含已知“正确答案”）
2. 边界 / 极值（1900-01-01、闰年2-29、Y10K 等）
3. 异常 / 非法输入（格式、范围、类型）
4. 月份 / 天数越界（13 月、32 日、负数）

把代码保存为 `test_date_weekday.py`，与附件 `mymodule.py` 同目录后执行 `pytest -q` 即可。

大模型辅助代码测试

• Step 2. 准备测试代码

- 将大模型给出的测试代码test_date_to_weekday.py下载到mymodule.py所在目录

```
45 INVALID_FORMAT = [  
46     # 无法按 'YYYY-MM-DD' 拆分成 3 个整数的情况  
47     ("2023/01/01",),          # 分隔符错误  
48     ("2023-1",),             # 缺少字段  
49     ("2023-01-01-02",),      # 多余字段  
50     ("",),                   # 空串  
51     ("2023-01-01 ",),        # 尾部空白  
52     (" 2023-01-01",),        # 前导空白  
53     ("abcd-ef-gh",),         # 非数字  
54     ("2023-01-01abc",),  
55 ]  
56  
57 @pytest.mark.parametrize("date_str, expected", VALID_DATES)  
58 def test_valid_dates(date_str, expected):  
59     assert date_to_weekday(date_str) == expected  
60  
61 @pytest.mark.parametrize("date_str", INVALID_FORMAT)  
62 def test_invalid_format(date_str):  
63     assert date_to_weekday(date_str) == "Invalid date format. Please use 'YYYY-MM-DD'."
```

当输入为非法日期时，目标函数必须返回“Invalid date format...”才能通过第63行的测试点



大模型辅助代码测试

pytest安装: `pip install pytest`

- Step 3. 运行测试代码

- 在命令行中进入上述目录, 执行 `pytest -q test_date_weekday.py`

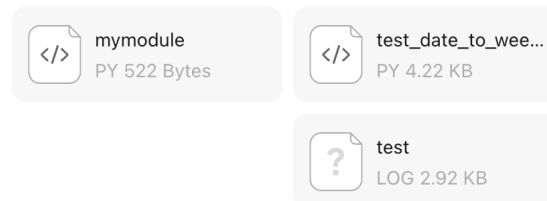
```
FAILED test_date_to_weekday.py::test_none_input - Failed: DID NOT RAISE AttributeError
FAILED test_date_to_weekday.py::test_int_input - Failed: DID NOT RAISE AttributeError
FAILED test_date_to_weekday.py::test_tuple_input - Failed: DID NOT RAISE AttributeError
FAILED test_date_to_weekday.py::test_bytes_input - Failed: DID NOT RAISE AttributeError
FAILED test_date_to_weekday.py::test_nonexistent_feb30 - [XPASS(strict)] 未校验日历合法性: 2 月 30 日不存在, 却被当作合法日期返回星期
FAILED test_date_to_weekday.py::test_month_13 - [XPASS(strict)] 未校验月份范围: 13 月被接受
FAILED test_date_to_weekday.py::test_month_0 - [XPASS(strict)] 未校验月份范围: 0 月被接受
FAILED test_date_to_weekday.py::test_before_min_date - [XPASS(strict)] 未校验日期范围: 小于 1900-01-01 的最小日期未拒绝
FAILED test_date_to_weekday.py::test_short_components - [XPASS(strict)] 未强制 YYYY 四位年份: '23-1-1' 被当作公元 23 年处理
9 failed, 38 passed in 0.03s
```

下一步把测试结果保存为测试日志 (test.log) 返回给大模型, 让其帮助调试目标函数, 并给出正确的实现



大模型辅助代码调试

- 原理：通过综合分析目标函数、测试用例、测试日志等多种信息，帮助开发者定位错误并提供修改建议



你是一个Python程序调试专家，善于根据代码的测试结果，发现并修改程序中的不完善之处。

下面是我写的代码和测试信息。请帮我检查并调试，找出其中的错误，给出最终改进后的代码。

+ 目标函数date_to_weekday位于mymodule.py

+ 测试脚本为test_date_to_weekday.py

+ 测试日志为test.log

在确保代码正确的前提下，代码尽可能简洁高效（最好不超过50行），且要包含必要的注释。

使用 7 个工具，Finalize mymodule.py Clean Review Deliver

验证日期格式并抛出错误

我先分析一下测试日志，找出问题所在，然后修复并实际运行测试验证。

Finalize mymodule.py Clean Review Deliver

运行 Python 代码

- Fix module to raise AttributeError for non-string split input

运行 Python 代码





大模型辅助代码调试

• Kimi给出的诊断及修改方案

问题诊断

`mymodule.py` 的 4 个缺陷:

1. 裸 `except`: 吞掉所有异常 —— `None / int` 等没有 `.split` 应抛 `AttributeError`, 却被捕获后返回了提示信息, 导致 4 个异常测试失败。
2. 无格式校验 —— 年份不强制 4 位 (`"23-1-1"` 被当成公元 23 年)。
3. 无日历校验 —— 2 月 30 日、第 13 月等不存在的日期被直接套进 Zeller 公式算出"星期"。
4. 无最小日期限制 —— 早于 1900-01-01 的日期未拒绝。

`test_date_to_weekday.py` 的 1 个隐藏 bug:

`INVALID_FORMAT` 写成了一元组列表 `[("2023/01/01"), ...]`。 `pytest.mark.parametrize` 对单参数不会拆包元组, 实际传入的是整个元组。原实现因裸 `except`: 吞掉 `AttributeError` 才"碰巧通过"; 修正实现后暴露出来。已改为纯字符串列表。

修复思路

用 `re.fullmatch(r"(\d{4})-(\d{2})-(\d{2})", ...)` 严格校验格式, 再用标准库 `datetime.date` 做日历合法性校验 (比手算闰年逻辑更可靠、代码更短), 星期名直接取 `date.weekday()` 查表, 不再手算 Zeller 公式。错误处理分三类, 与测试约定严格对齐:

```
10 import re
11 from datetime import date
12
13 MIN_DATE = date(1900, 1, 1) # 支持的最小日期
14 INVALID_MSG = "Invalid date format. Please use 'YYYY-MM-DD'."
15 # date.weekday() 的返回值: 0=Monday ... 6=Sunday
16 _WEEKDAYS = ["Monday", "Tuesday", "Wednesday", "Thursday",
17              "Friday", "Saturday", "Sunday"]
18
19
20 def date_to_weekday(date_str):
21     """返回 date_str 对应的英文星期名; 非法输入按模块 docstring 的约定处理。"""
22     if not isinstance(date_str, str):
23         # None/int/tuple 无 .split, bytes.split 则抛 TypeError; 统一为 AttributeError
24         raise AttributeError("date_str must be a str, got %r" % type(date_str).__name__)
25     m = re.fullmatch(r"(\d{4})-(\d{2})-(\d{2})", date_str)
26     if m:
27         y, mo, d = map(int, m.groups())
28         dt = date(y, mo, d) # 日历非法 (如 2 月 30 日、13 月) 在此抛出 ValueError
29         if dt < MIN_DATE:
30             raise ValueError("dates before 1900-01-01 are not supported")
31         return _WEEKDAYS[dt.weekday()]
32     # 能拆成 3 个整数但位数不对 (如 '23-1-1'): 属越界输入, 抛错而非返回提示
33     if re.fullmatch(r"\d+-\d+-\d+", date_str):
34         raise ValueError("date components must be zero-padded, e.g. 'YYYY-MM-DD'")
35     return INVALID_MSG
```



大模型辅助代码调试

- 测试Kimi方案
 - 将Kimi给出的方案保存为mymodule_kimi.py
 - 将之前的测试代码复制为test_date_to_weekday_kimi.py并修改
 - `from mymodule_kimi import date_to_weekday`
 - 执行命令 `pytest -q test_date_to_weekday_kimi.py`

```
FAILED test_date_to_weekday_kimi.py::test_invalid_format[date_str0] - AttributeError: date_str must be a str, got 'tuple'
FAILED test_date_to_weekday_kimi.py::test_invalid_format[date_str1] - AttributeError: date_str must be a str, got 'tuple'
FAILED test_date_to_weekday_kimi.py::test_invalid_format[date_str2] - AttributeError: date_str must be a str, got 'tuple'
FAILED test_date_to_weekday_kimi.py::test_invalid_format[date_str3] - AttributeError: date_str must be a str, got 'tuple'
FAILED test_date_to_weekday_kimi.py::test_invalid_format[date_str4] - AttributeError: date_str must be a str, got 'tuple'
FAILED test_date_to_weekday_kimi.py::test_invalid_format[date_str5] - AttributeError: date_str must be a str, got 'tuple'
FAILED test_date_to_weekday_kimi.py::test_invalid_format[date_str6] - AttributeError: date_str must be a str, got 'tuple'
FAILED test_date_to_weekday_kimi.py::test_invalid_format[date_str7] - AttributeError: date_str must be a str, got 'tuple'
8 failed, 34 passed, 5 xfailed in 0.04s
```

还是没有通过测试 ☹

大模型辅助代码调试

不能对大模型给出的方案
照单全收，要“以我为主”

- 原因：目标函数与测试代码关于非法输入的处理逻辑不一致
 - 目标函数抛出错误（第24行）
 - 测试代码期待特殊文本（第63行）

```
10 import re
11 from datetime import date
12
13 MIN_DATE = date(1900, 1, 1) # 支持的最小日期
14 INVALID_MSG = "Invalid date format. Please use 'YYYY-MM-DD'."
15 # date.weekday() 的返回值: 0=Monday ... 6=Sunday
16 _WEEKDAYS = ["Monday", "Tuesday", "Wednesday", "Thursday",
17              "Friday", "Saturday", "Sunday"]
18
19
20 def date_to_weekday(date_str):
21     """返回 date_str 对应的英文星期名；非法输入按模块 docstring 的约定处理。"""
22     if not isinstance(date_str, str):
23         # None/int/tuple 无 .split, bytes.split 则抛 TypeError; 统一为 AttributeError
24         raise AttributeError("date_str must be a str, got %r" % type(date_str).__name__)
25     m = re.fullmatch(r"(\d{4})-(\d{2})-(\d{2})", date_str)
26     if m:
27         y, mo, d = map(int, m.groups())
28         dt = date(y, mo, d) # 日历非法 (如 2 月 30 日、13 月) 在此抛出 ValueError
29         if dt < MIN_DATE:
30             raise ValueError("dates before 1900-01-01 are not supported")
31         return _WEEKDAYS[dt.weekday()]
32     # 能拆成 3 个整数但位数不对 (如 '23-1-1') : 属越界输入, 抛错而非返回提示
33     if re.fullmatch(r"\d+-\d+-\d+", date_str):
34         raise ValueError("date components must be zero-padded, e.g. 'YYYY-MM-DD'")
35     return INVALID_MSG
```

```
45 INVALID_FORMAT = [
46     # 无法按 'YYYY-MM-DD' 拆分成 3 个整数的情况
47     ("2023/01/01",), # 分隔符错误
48     ("2023-1",), # 缺少字段
49     ("2023-01-01-02",), # 多余字段
50     ("",), # 空串
51     ("2023-01-01 ",), # 尾部空白
52     (" 2023-01-01",), # 前导空白
53     ("abcd-ef-gh",), # 非数字
54     ("2023-01-01abc",),
55 ]
56
57 @pytest.mark.parametrize("date_str, expected", VALID_DATES)
58 def test_valid_dates(date_str, expected):
59     assert date_to_weekday(date_str) == expected
60
61 @pytest.mark.parametrize("date_str", INVALID_FORMAT)
62 def test_invalid_format(date_str):
63     assert date_to_weekday(date_str) == "Invalid date format. Please use 'YYYY-MM-DD'."
```

修改测试代码

```
61
62 @pytest.mark.parametrize("date_str", INVALID_FORMAT)
63 def test_invalid_format(date_str):
64     # assert date_to_weekday(date_str) == "Invalid date format. Please use 'YYYY-MM-DD'."
65     with pytest.raises(AttributeError):
66         date_to_weekday(date_str)
67
```



大模型辅助代码调试

• 对比不同大模型给出的修改方案

Kimi

```
10 import re
11 from datetime import date
12
13 MIN_DATE = date(1900, 1, 1) # 支持的最小日期
14 INVALID_MSG = "Invalid date format. Please use 'YYYY-MM-DD'."
15 # date.weekday() 的返回值: 0=Monday ... 6=Sunday
16 _WEEKDAYS = ["Monday", "Tuesday", "Wednesday", "Thursday",
17              "Friday", "Saturday", "Sunday"]
18
19
20 def date_to_weekday(date_str):
21     """返回 date_str 对应的英文星期名; 非法输入按模块 docstring 的约定处理。"""
22     if not isinstance(date_str, str):
23         # None/int/tuple 无 .split, bytes.split 则抛 TypeError; 统一为 AttributeError
24         raise AttributeError("date_str must be a str, got %r" % type(date_str).__name__)
25     m = re.fullmatch(r"(\d{4})-(\d{2})-(\d{2})", date_str)
26     if m:
27         y, mo, d = map(int, m.groups())
28         dt = date(y, mo, d) # 日历非法 (如 2 月 30 日、13 月) 在此抛出 ValueError
29         if dt < MIN_DATE:
30             raise ValueError("dates before 1900-01-01 are not supported")
31         return _WEEKDAYS[dt.weekday()]
32     # 能拆成 3 个整数但位数不对 (如 '23-1-1'): 属越界输入, 抛错而非返回提示
33     if re.fullmatch(r"\d+-\d+-\d+", date_str):
34         raise ValueError("date components must be zero-padded, e.g. 'YYYY-MM-DD'")
35     return INVALID_MSG
```

DeepSeek

```
1 def date_to_weekday(date_str):
2     """将 'YYYY-MM-DD' 日期字符串转换为英文星期名 (Zeller 公式) 。"""
3     if not isinstance(date_str, str):
4         raise AttributeError("date_str must be a string")
5     parts = date_str.split('-')
6     # 必须是 3 段且每段为纯数字, 否则视为格式错误
7     if len(parts) != 3 or not all(p.isdigit() for p in parts):
8         return "Invalid date format. Please use 'YYYY-MM-DD'."
9
10    year, month, day = map(int, parts)
11    # 1、2 月按上一年的 13、14 月处理
12    if month < 3:
13        year -= 1
14        month += 12
15
16    q, m = day, month
17    k, j = year % 100, year // 100
18    h = (q + 13 * (m + 1) // 5 + k + k // 4 + j // 4 + 5 * j) % 7
19    weekdays = ["Saturday", "Sunday", "Monday", "Tuesday",
20               "Wednesday", "Thursday", "Friday"]
21    return weekdays[h]
```



课堂练习-7-3

- 用大模型给练习7-2编写的密码生成器生成一组pytest测试用例
 - 编写提示词
 - 用pytest测试
 - 若测试未通过，是之前的实现有bug，还是测试用例过严？
 - 把测试日志发给大模型追问



大模型辅助编程总结

- 一个原则
 - 大模型是辅助而非替代
- 四个角色
 - 老师（概念讲解） · 打字员（代码生成补全） · 医生（诊断调试） · 评论员（解释重构）
- 提示词四要素
 - 角色设定 · 任务描述 · 约束与要求 · 输入输出示例

始终“以我为主”：注意数据安全和隐私，对大模型给出的方案要加以甄别